BRAIN FINANCE INC.

MVP Architecture and Technical Plan

The programmable bank and marketplace for AI agents

1. Overview

Purpose of This Document

This document sets out the complete MVP architecture for Brain, a programmable bank and marketplace for AI agents. It covers the MVP scope and rationale, the full system architecture including all partner integrations, the account setup and agent deployment flow, the x402 payment execution flow, and the AI reasoning layer that powers agent intelligence. It is intended for investors, technical advisors, and the founding team.

What Brain Is

Brain is building the financial infrastructure layer for the agent economy. It combines a stablecoin neobank, an agent execution framework, and a developer marketplace into a single programmable financial environment. The platform operates at two levels:

Layer	Description
Brain Platform	Consumer-facing financial environment: stablecoin accounts, agent deployment, developer marketplace, debit card.
Brain Protocol	Open infrastructure standard built on ERC-4337, ERC-8004, x402, and related standards. Independently deployable by any developer.

What the MVP Must Prove

The investor demo loop is:

1. A user signs up
2. Deposits USDC
3. Creates an agent with a spend policy
4. The agent autonomously executes a payment to an external service via x402.

The MVP does not need to be a full product. It needs to prove that the core loop works end to end: capital in, agent deployed, agent acts, policy enforces, payment settles on-chain, receipt recorded.

2. MVP Scope

In Scope

- Stablecoin account (USDC deposit, balance, transfer) via Coinbase on-ramp
- Agent sub-account with isolated capital pool per agent
- Policy configuration: spend limit, allowed assets, time window, approval threshold
- BrainAccount smart contract (ERC-4337) deployed on Base
- PolicyValidator contract with on-chain policy proof verification
- ERC-8004 agent registry: agentId assignment, status, validation records
- x402 payment flow: parse 402 response, policy check, on-chain settlement, receipt
- One prebuilt agent template (payment agent, simplest flow)
- AI reasoning layer: Claude tool use, ReAct agent loop, three-tier memory
- WireX bank account and card provisioning (runs in parallel, not blocking the demo)

Deferred to Phase 2

- IBAN accounts and fiat rails
- KYC integration (Sumsub)
- Trading integration (Hyperliquid, Polymarket)
- Lending and credit products (Aave, Compound)
- Agent swarms and multi-agent coordination
- Developer marketplace
- EIP-7702 EOA delegation
- Subscription billing (premium swarm tier)

Timeline

Phase	What	Duration
Smart contracts and protocol	ERC-4337 account, PolicyValidator, ERC-8004 registry	TBD
Banking and on-ramp	USDC account, Coinbase on-ramp, WireX provisioning	TBD
AI reasoning layer	Claude tool use, ReAct loop, pgvector memory	TBD
x402 payment flow	Payment Orchestrator, single demo endpoint	TBD
Agent template and UI wiring	Policy config, dashboard, deploy flow	TBD
Testnet alpha	Internal demo, audit submission, bug fixes	TBD

3. System Architecture

Layer Overview

The MVP architecture is structured across seven distinct layers. Each layer validates independently while composing into a single deterministic execution pipeline. No action can bypass identity verification, policy enforcement, or payment validation.

Client	Brain web app, mobile app, SDK. Users configure agent goals, review policy, monitor activity.
AI Reasoning	LLM reasoning (Claude / GPT-4o via tool use), ReAct agent loop (reason, act, observe), three-tier memory (context, vector, structured), tool registry.
Identity	SIWX authentication for users and agents. ERC-8004 agent registry assigning persistent agentId.
Control Plane	Policy Engine (evaluate, sign proof), Agent Service (lifecycle, config), Payment Orchestrator (x402 flow, UserOp assembly).
On-Chain	BrainAccount ERC-4337 smart account, PolicyValidator contract, Alchemy bundler (ERC-7769).
Partners	Coinbase Embedded Wallets (user wallet, on/off-ramp), Coinbase Agentic Wallets (agent execution signing), WireX (bank account, card).

Infrastructure

Component	Provider	Role
Execution network	Base	Primary chain for all agent transactions
Identity anchor	Ethereum	ERC-8004 registry, high-value settlement
RPC and bundler	Alchemy	UserOperation submission, ERC-7769 bundler
Vector memory	pgvector (Postgres)	Short-term agent memory for recent observations
LLM API	Anthropic (Claude) + OpenAI backup	Agent reasoning and tool use
AI services	OpenAI, Anthropic	x402 demo target endpoints

4. Partner Integrations

Coinbase Embedded Wallets

Coinbase Embedded Wallets owns the user side of the capital stack. It provisions the user's primary wallet on sign-up, handles private key management so Brain never touches user keys, and provides the on/off-ramp so users can deposit USDC from fiat without leaving the Brain app.

- Provisions user wallet at sign-up
- Manages private keys; Brain never holds user keys
- Provides USDC on-ramp from fiat
- Serves as the owner address of the BrainAccount smart contract

> *Priority: Critical path. Coinbase Embedded Wallet must be live before BrainAccount deployment can be wired to a real user key. Initiate sandbox access in week 1.*

Coinbase Agentic Wallets

Coinbase Agentic Wallets is a separate wallet primitive designed specifically for autonomous execution. Each agent gets its own Agentic Wallet as its execution address. This wallet signs UserOperations and executes x402 payments on-chain.

- Each deployed agent receives its own Agentic Wallet as its execution address
- Signs UserOperations submitted to the ERC-4337 EntryPoint
- Isolated from the user's primary wallet by architecture
- Can only access capital explicitly allocated via BrainAccount authorizeAgent

> *Priority: Critical path. Coinbase Agentic Wallet must be live before any agent can sign a UserOperation. Initiate sandbox access in week 1, in parallel with Embedded Wallets.*

WireX

WireX operates in the fiat and card layer, parallel to the crypto stack rather than inside it. It provisions the stablecoin account and IBAN account and issues the debit card. For the MVP it provides the banking identity and card issuance layer.

- Provisions stablecoin account (USDC-denominated)
- Issues IBAN account for fiat connections
- Issues debit card linked to account balances
- Operates independently of the smart account and agent execution flow

Priority: Not blocking the investor demo. WireX handles off-chain financial rails. The x402 demo can function with Coinbase and on-chain components while WireX integration is being completed. Initiate business agreement in week 1 but do not let it gate the demo milestone.

Integration Dependency Order

Integration	Blocks what	Priority
Coinbase Embedded Wallets	User sign-up, USDC deposit, BrainAccount owner address	Critical path, week 1
Coinbase Agentic Wallets	Agent execution, x402 payments, UserOp signing	Critical path, week 1
Alchemy RPC and bundler	All on-chain transactions	Critical path, week 1
WireX bank and card	Card spending, IBAN rails	Non-blocking

5. Account Setup and Agent Deployment

Account Setup Flow

The account setup flow runs left to right, establishing the user's financial environment before any agent is deployed.

1. User registers via SIWX sign-in. Email and cryptographic signature create a scoped session.
2. Coinbase Embedded Wallet is provisioned. The user's primary wallet is created with key management handled by Coinbase.
3. WireX account is provisioned. A stablecoin account and debit card are created in parallel.
4. BrainAccount smart contract is deployed on Base. The user's Coinbase wallet address becomes the owner.
5. User deposits USDC via the Coinbase on-ramp. Funds land in the BrainAccount.
6. Primary account is funded and ready for agent allocation.

Agent Deployment Flow

Agent deployment runs in parallel with account setup. Once both are complete, capital is allocated to the agent and it goes live.

7. User configures a policy: spend limit, allowed assets, time window, approval threshold.
8. Policy hash is written on-chain via setPolicy on the BrainAccount.
9. A Coinbase Agentic Wallet is created as the agent's execution address.
10. The agent is registered in the ERC-8004 registry. A persistent agentId is assigned.
11. The agent is authorised on the BrainAccount via authorizeAgent. It is now scoped to its policy.
12. Capital is allocated from the primary account to the agent's sub-account. The agent is live.

Key Architecture Principle

> *An agent can only act within the capital explicitly allocated to its sub-account. It has no visibility into and no access to the user's primary account or any other agent's allocation. This separation is enforced at the smart contract level, not by application logic.*

6. x402 Payment Execution Flow

Overview

x402 is the protocol for machine-to-machine payments over standard HTTP. When an agent requests a paid resource, the server responds with a 402 Payment Required response and a structured payment header. Brain handles policy evaluation and on-chain settlement automatically. Agents and servers communicate only via standard HTTP headers.

Step-by-Step Flow

13. Agent sends GET /resource with Agent-ID header via its Coinbase Agentic Wallet identity.
14. Server returns 402 with X-402-Payment header: amount, asset, merchant address, expiry, nonce.
15. Payment Orchestrator parses the header and constructs a PaymentIntent: agentId, resourceUri, amount, asset, merchant, expiry, nonce.
16. Policy Engine evaluates the PaymentIntent: budget check, asset allowlist, merchant allowlist, frequency limit. Issues a signed policy proof if approved.
17. UserOperation is assembled with the policy proof attached and submitted to the Alchemy bundler.
18. Alchemy bundler submits to the ERC-4337 EntryPoint. PolicyValidator verifies the proof on-chain. BrainAccount executes the USDC transfer via the Coinbase Agentic Wallet.
19. Receipt Indexer records the result: agentId, PaymentIntent, on-chain tx hash, HTTP correlation ID.
20. ERC-8004 validation record is written to the agent registry.
21. Brain retries the original HTTP request with X-402-Receipt header. Server verifies and serves the resource.

Security Guarantees

Mechanism	What it Prevents
Nonces	Replay attacks on payment intents
Expiries	Stale or delayed charge execution
Per-merchant budgets	Overspending to a single counterparty
PolicyValidator (on-chain)	Execution without a valid signed policy proof
Receipt binding	Disputes: every payment links HTTP request to on-chain tx

7. AI Reasoning Layer

Recommendation: Model-Agnostic Runtime with Thin Abstraction

The recommended approach is to avoid hardcoding a single LLM provider into agent logic. Brain's value proposition is the financial infrastructure layer, not the AI layer. The intelligence must be pluggable. The recommended stack has three components.

Component 1: LLM Reasoning via Tool Use

Anthropic's Claude with tool use (function calling) is the recommended default. Tool use is the right primitive for financial agents: the LLM reasons about when to call Brain's execution functions (swap, transfer, pay, check_balance) rather than generating free-form text. The agent reasons, decides to call a tool, Brain executes within policy, and the result goes back to the model.

- Default: Claude via the Anthropic API with tool use
- Secondary: OpenAI GPT-4o (identical tool use patterns, same abstraction layer)
- The LLM proposes. The policy engine decides. The Coinbase Agentic Wallet executes.

> *A hallucinating model, a jailbroken prompt, or a malicious agent template cannot bypass financial controls. The smart account and PolicyValidator enforce constraints independently of whatever the LLM decides.*

Component 2: ReAct agent loop

The agent loop is a lightweight custom implementation of the ReAct pattern: Reason, Act, Observe. This sits server-side in Brain's backend between the client request and the control plane.

22. LLM receives: agent objective, current account state, tool list, memory context.
23. LLM reasons and returns a structured tool call.
24. Loop passes the tool call to the Brain control plane as a proposed action.
25. Policy Engine evaluates. If approved: Coinbase Agentic Wallet executes. If rejected: rejection reason returned to LLM as observation.
26. The result feeds back to LLM as an observation. The loop continues until the objective is complete.

For the MVP, a lightweight custom loop is cleaner than a heavyweight framework and gives more control over exactly when tools are called and how policy proofs are attached. LangChain or LlamaIndex can be substituted as complexity grows.

Component 3: Three-Tier Memory Architecture

Tier	Storage	Contents	Scope
In-context	LLM prompt window	Current objective, last few observations, active position	Current session only
Short-term	pgvector (Postgres)	Recent transaction embeddings, market observations, conversation history	Retrieved by semantic similarity
Long-term structured	Agent database record	Strategy parameters, risk tolerance, persistent state, running totals	Persists across all sessions

At the start of each agent turn, a retrieval step queries pgvector for the most semantically relevant recent observations. These are injected into the LLM context alongside the agent's current objective and account state. The LLM does not need to remember every transaction; it needs the current position and the most relevant recent events.

Component 4: Tool Registry

Every function the LLM is allowed to call is declared as a JSON tool schema, as Claude and OpenAI's tool use APIs expect. Brain controls this list entirely.

Tool	What it Does
transfer()	Move USDC between accounts within policy limits
pay_x402()	Execute an x402 payment to an external service
check_balance()	Read current balance of the agent's sub-account
get_policy()	Retrieve the agent's active policy configuration
record_action()	Write an observation or decision to agent memory

> *Adding a tool means adding a new callable action in the control plane. Removing a tool means the LLM cannot attempt that action regardless of what the user or a prompt injection attempts.*

MVP AI configuration

For the investor demo, the simplest possible configuration is sufficient:

- One agent type: payment agent
- One LLM: Claude via the Anthropic API with three registered tools (pay_x402, check_balance, get_policy)
- One memory store: pgvector table with the last 20 transactions as short-term memory
- One demo target: an Anthropic or OpenAI API endpoint that accepts x402 payment headers

This is enough to show an agent reasoning about whether to execute a payment, passing the decision through the policy engine, and settling on-chain.